RoadSense: Robust Traffic Sign Recognition

Benchmarking scratch-trained and fine-tuned CNNs on GTSRB under image corruptions, with Grad-CAM explainability

Group 4 | Atria University | Neural Networks and CNN Sprint | Instructor: Aishwarya Mishra | November 2026

Public GitHub repository: https://github.com/rajukumar-tech/RoadSense-Robust-Traffic-Sign-Recognition

Member	GitHub	Role	Contribution (as recorded in the repository README)
Aseem (Team Leader)	mohammedasim-ee	Leader; Phase 6	Dataset acquisition (src/download_data.py), project coordination, pipeline architecture, documentation, this report
Raju	rajukumar-tech	Phases 1-3	EDA, leakage-free track split, scratch CNN, training loop, clean evaluation, corruption robustness bench
Manoj	MAN_OF_HATERS	Phase 4 (code)	Transfer-learning model class (ResNet-18, MobileNetV2), unified training pipeline, two-stage fine-tuning, evaluation extensions
Sohail	Sohail-WD	Phase 4 (training), Phase 5	Training and evaluation of Phase 4 models, Grad-CAM explainability and interpretation

Declaration of AI-assistant use. An AI assistant (Claude, Anthropic) was used to (i) draft and typeset this report from the repository results, (ii) generate the report figures from the committed CSV files, and (iii) write the Gradio demo interface (app.py). All numbers in this report were read from the CSV/JSON files in the repository and cross-checked by the assistant against the source code. Use of AI tools by other members during coding is to be declared by each member; see Section 10.

Abstract. Traffic-sign classifiers score above 97% on the clean GTSRB test set, yet cameras on real roads produce noise, blur, glare, occlusion and compression artefacts that the clean benchmark never shows. RoadSense asks which of four convolutional models stays reliable when images degrade. We compare a scratch-trained CNN (823k parameters), the same CNN trained with label-safe augmentation, and ImageNet-pretrained ResNet-18 and MobileNetV2 fine-tuned in two stages. To avoid leakage, validation images are split by physical sign track (1,046 train / 261 validation tracks, zero overlap). Each model is evaluated on the full 12,630-image test set and on 15 corruption conditions (5 corruption types x 3 severities); we summarise robustness as mean corrupted accuracy (mCA). The augmented scratch CNN is best on both axes (98.70% clean, 83.11% mCA), the baseline follows (97.98%, 81.46%), then ResNet-18 (96.27%, 73.82%) and MobileNetV2 (93.54%, 66.18%). Augmentation helped four of five corruption types but not Gaussian noise, and pretrained models were far more noise-sensitive. Grad-CAM was used for qualitative inspection; the evidence is limited, and we state the limits. The lesson: clean accuracy hides large robustness differences.

1. Introduction and Motivation

Driver-assistance and autonomous systems must read traffic signs reliably in all conditions. A misread speed limit or a missed stop sign is a safety issue, not just an accuracy loss. The German Traffic Sign Recognition Benchmark (GTSRB) is the standard dataset for this task, and modern CNNs reach about 98-99% on its test set, which makes clean accuracy a poor way to tell models apart. The gap that matters for deployment is what happens when the camera image is worse than the benchmark image: sensor noise at night, motion blur, glare or shadow, a sticker or branch covering part of the sign, and heavy compression in transmitted video.

This project therefore treats **robustness** as the main question. We ask: (1) how accurate are different CNN families on clean data; (2) how quickly does each degrade under controlled corruptions; (3) does label-safe augmentation or ImageNet pretraining improve robustness; and (4) can Grad-CAM help us understand failures? The report follows the course brief: dataset and leakage-free validation, baseline, robustness benchmark, improvement, explainability, and a candid discussion of limitations.

Contributions. (a) A track-based, leakage-free split of GTSRB; (b) a reproducible corruption benchmark with a single summary metric (mCA) applied to all 12,630 test images; (c) a controlled four-model comparison; (d) an honest analysis of what the evidence does and does not support; and (e) an interactive demo (app.py) that shows predictions and Grad-CAM for clean and corrupted images.

2. Related Work

GTSRB and the "human-level" result. Stallkamp et al. (2012) released GTSRB and showed that a committee of CNNs reached 99.46% accuracy, above the 98.84% of human volunteers on the same images. This established that clean GTSRB is nearly saturated, which is why we focus on degradation rather than only clean accuracy.

Multi-scale and compact CNNs. Sermanet and LeCun (2011) fed features from several stages of a convolutional network to the classifier and reached 98.97%, showing that a small CNN trained from scratch is enough for this dataset. This supports our choice of a compact scratch baseline and our finding that larger pretrained models are not automatically better.

Robustness to common corruptions. Hendrycks and Dietterich (2019) introduced ImageNet-C, a benchmark of 15 corruption types at five severities, and reported mean corruption accuracy as a summary. Our benchmark follows the same idea at smaller scale (5 types, 3 severities) and uses mCA as the headline robustness number.

Transfer learning. He et al. (2016) introduced residual connections (ResNet), and Sandler et al. (2018) introduced the inverted residual block of MobileNetV2 for efficient inference. Both are commonly fine-tuned from ImageNet weights; we test whether this helps 43-class signs at low resolution.

Explainability. Selvaraju et al. (2017) proposed Grad-CAM, which weights the last convolutional feature maps by the gradient of the class score to give a coarse localisation map. We use it as a qualitative aid and not as proof of causal reasoning.

3. Dataset

We use GTSRB: **39,209** training images and **12,630** test images across **43** classes, cropped to the sign region by the dataset annotations. Image sides range from about 25 to 243 pixels with a strong peak around 30-50 pixels (the repository EDA figure), so all models work on heavily downsampled inputs. The class distribution is imbalanced: the largest class has 2,250 images and the smallest 210, a ratio of **10.71**.

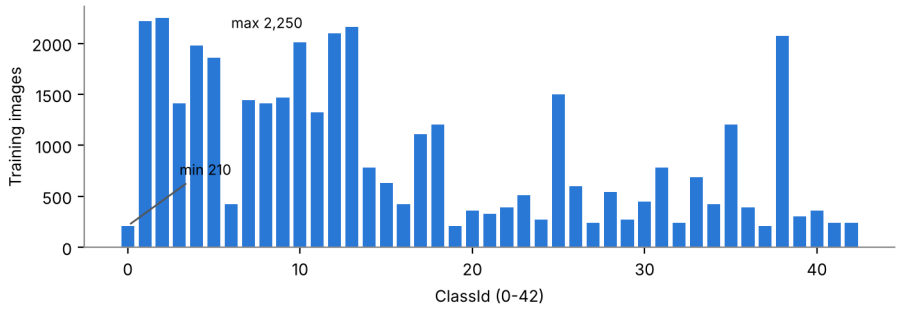


Figure 1. Training images per class (Train.csv). Several speed-limit classes dominate; the rarest classes have only 210 images.

Leakage-free split. GTSRB images come in tracks of about 30 consecutive frames of the same physical sign. A random image-level split puts near-duplicate frames in both training and validation, which inflates validation accuracy. We therefore split by track: 1,307 tracks in total, **1,046 for training (31,379 images) and 261 for validation (7,830 images)**, with zero overlap, seed 42. The official test set is used only for the final evaluation.

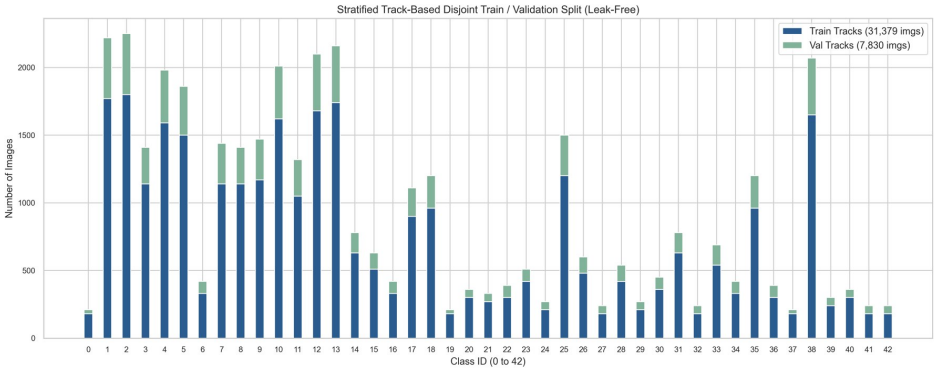


Figure 2. Track-based split: class counts in the training and validation sets (repository EDA output).

Preprocessing. Images are resized (32x32 for the scratch CNNs, 48x48 for the transfer models), converted to float tensors and normalised (ImageNet statistics for the pretrained models). The class imbalance is handled with a square-root-smoothed inverse-frequency class weight in the loss. **Augmentation is label-safe**: small rotation (10 degrees), translation (6%), scale (0.95-1.05) and colour jitter (0.2); horizontal and vertical flips are deliberately excluded, because a flipped "turn left" sign becomes a "turn right" sign and would corrupt the label.

4. Methodology

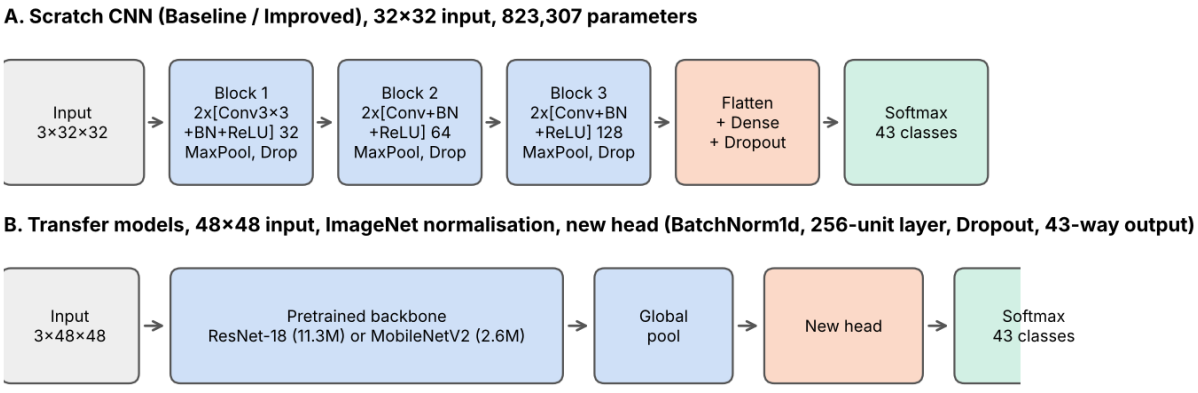


Figure 3. Architectures. (A) Scratch CNN used for baseline and improved models. (B) Pretrained backbones with a new classification head.

Models. The **baseline CNN** has three convolutional blocks (32, 64, 128 channels), each with two 3x3 convolutions, batch normalisation, ReLU, max-pooling and dropout, followed by a dense classifier:

823,307 parameters. The **improved CNN** is identical in architecture but trained with the label-safe augmentation above. **ResNet-18** (11,319,147 parameters) and **MobileNetV2** (2,563,115 parameters) start from ImageNet weights with a new head of batch normalisation, a 256-unit layer, dropout and a 43-way output.

Training. Loss: class-weighted cross-entropy. Optimiser: Adam, weight decay 1e-4, initial learning rate 1e-3, batch size 64, ReduceLROnPlateau on validation accuracy (factor 0.5, patience 2). The checkpoint with the best validation accuracy is kept. For the pretrained models we use **two-stage fine-tuning**: 5 epochs with the backbone frozen (head only), then full fine-tuning for the remaining epochs with learning rate 1e-4 for the backbone and 1e-3 for the head. The improved CNN and both transfer models were trained for 15 epochs; the per-epoch history of the baseline contains 8 epochs, so the baseline and improved CNN differ in training length as well as augmentation (see Section 7).

Setting	Baseline CNN	Improved CNN	ResNet-18	MobileNetV2
Input size	32x32	32x32	48x48	48x48
Parameters	823,307	823,307	11,319,147	2,563,115
Initialisation	random	random	ImageNet	ImageNet
Augmentation	none	label-safe	label-safe	label-safe
Epochs in history	8	15	15	15
Fine-tuning	n/a	n/a	5 frozen + full	5 frozen + full

Table 1. Model configurations (from src/models.py, src/train.py and the saved history files).

Hyper-parameters.

Hyper-parameter	Value	Applies to
Optimiser	Adam, weight decay 1e-4	all models
Initial learning rate	1e-3 (stage 2: backbone 1e-4, head 1e-3)	all / transfer stage 2
LR schedule	ReduceLROnPlateau on val. accuracy, factor 0.5, patience 2	all models
Batch size	64	all models
Loss	Cross-entropy, sqrt inverse-frequency class weights	all models
Frozen-backbone stage	5 epochs (head only)	ResNet-18, MobileNetV2
Random seed	42 (split, corruptions)	all
Rotation / translation / scale / colour jitter	10 deg / 6% / 0.95-1.05 / 0.2	improved CNN and transfer models

Table 1b. Training hyper-parameters (src/train.py defaults and docs/STEP5; the README also describes an alternative optimised runner, run_fast.py, with different settings; the results reported here are those committed in results/).

Robustness benchmark. Five corruptions at three severities are applied on the fly to all 12,630 test images with a per-image seed (42 + image index), so every model sees exactly the same corrupted images. Gaussian noise (sigma 18, 38, 65 on a 0-255 scale); blur (Gaussian blur, radius 1.0, 2.0, 3.5; named "motion_blur" in the code, but it is isotropic blur, not directional motion blur); brightness/contrast reduction (contrast, brightness = 0.80/0.75, 0.60/0.50, 0.40/0.30); random grey occlusion (patch side 18.75%, 31.25%, 43.75% of the image); and JPEG compression (quality 40, 20, 8). **mCA** is the mean accuracy over the 15 conditions.

5. Experiments and Results

5.1 Clean test performance

Model	Clean acc. (%)	Macro-F1 (%)	Weighted-F1 (%)	Macro-P (%)	Macro-R (%)	mCA (%)
Baseline CNN	97.98	97.23	97.98	97.08	97.57	81.46
Improved CNN	**98.70**	**97.98**	**98.69**	97.86	98.18	**83.11**
ResNet-18	96.27	94.70	96.25	94.78	95.12	73.82
MobileNetV2	93.54	92.29	93.56	93.05	92.45	66.18

Table 2. Results on the full 12,630-image GTSRB test set (results/metrics.csv, results/model_comparison.csv).

The two scratch CNNs beat both pretrained models on clean data. A plausible reason is that ImageNet features are tuned for natural photos at 224 pixels, while GTSRB signs are small, simple, high-contrast shapes; we did not test this explanation directly, so it remains a hypothesis. All results are from a single training run per model with seed 42, so differences of a few tenths of a percent (for example 97.98 vs 98.70) should be read with caution: no confidence intervals were computed.

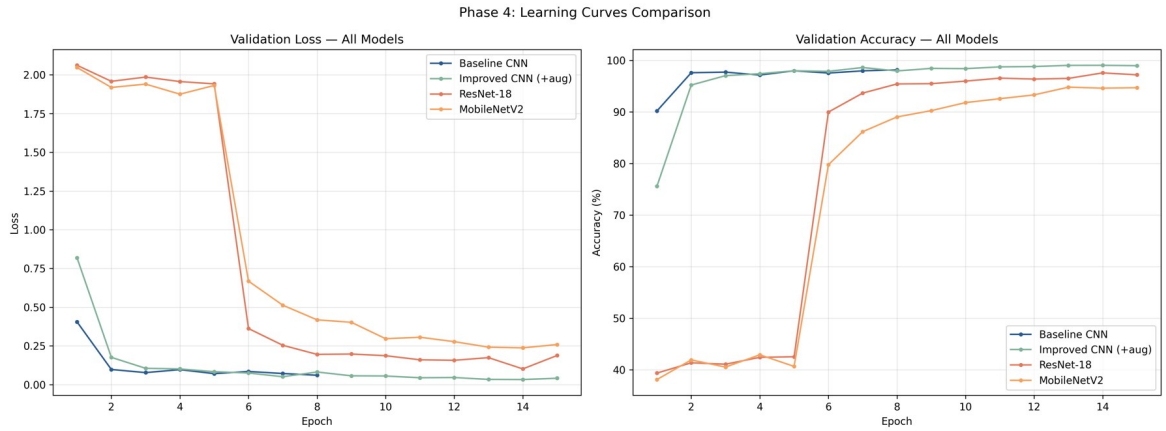


Figure 4. Training and validation curves for the four models (repository output, results/figures). The transfer models reach high training accuracy but show higher validation loss than the scratch CNNs (validation loss at last epoch: improved CNN 0.041, ResNet-18 0.188, MobileNetV2 0.259).

Model	Epochs logged	Final train acc. (%)	Final val. acc. (%)	Final train loss	Final val. loss	Best val. acc. (%)
Baseline CNN	8	99.17	98.12	0.033	0.060	98.12
Improved CNN	15	99.13	98.94	0.031	0.041	99.02
ResNet-18	15	99.38	97.16	0.022	0.188	97.55
MobileNetV2	15	98.81	94.67	0.038	0.259	94.78

*Table 2b. Training dynamics from results/*_history.json. The validation set is the track-disjoint split, so these numbers are comparable to test accuracy; the transfer models show a larger train-validation loss gap.*

5.2 Robustness under corruption

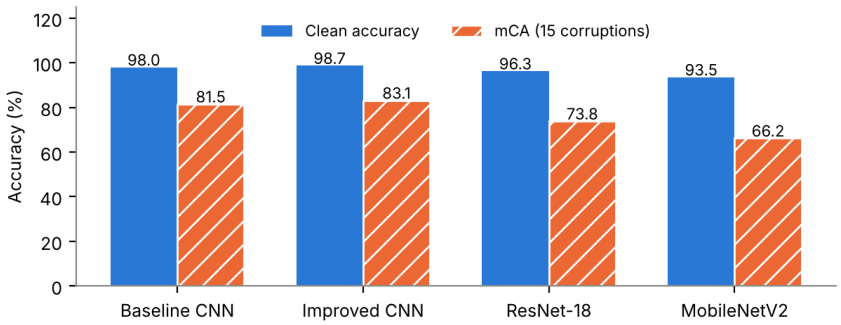


Figure 5. Clean accuracy versus mCA. Every model loses accuracy under corruption; the gap is smallest for the scratch CNNs (about 15 points) and largest for MobileNetV2 (about 27 points).

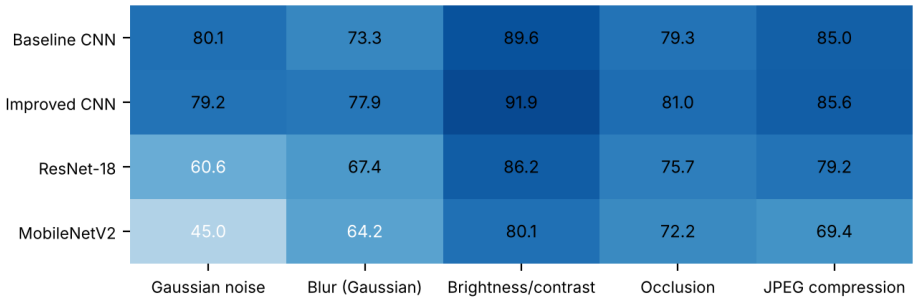


Figure 6. Mean accuracy per corruption type (average over 3 severities). Gaussian noise is the most damaging for the pretrained models (ResNet-18 60.6%, MobileNetV2 45.0%).

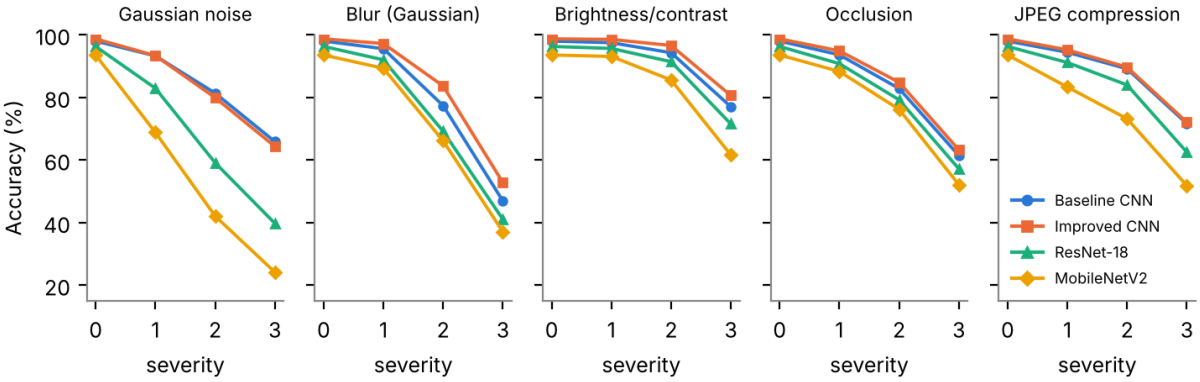


Figure 7. Accuracy against severity (0 = clean) for each corruption. Models are ordered the same way at nearly every point; the exceptions are Gaussian noise, where the baseline matches or exceeds the improved CNN.

Corruption	Sev.	Baseline	Improved	ResNet-18	MobileNetV2
Gaussian noise	1	93.24	**93.33**	82.91	68.80
	2	**81.35**	79.90	59.09	42.07
	3	**65.85**	64.32	39.75	24.16
Blur	1	95.55	**97.20**	92.00	89.29
	2	77.36	**83.64**	69.16	66.12
	3	46.90	**52.80**	41.03	37.07
Brightness/contrast	1	97.47	**98.52**	95.63	93.07
	2	94.26	**96.56**	91.39	85.55
	3	77.01	**80.60**	71.49	61.62
Occlusion	1	93.59	**94.86**	90.78	88.29
	2	82.83	**84.81**	79.22	76.18
	3	61.40	**63.24**	57.18	52.12
JPEG	1	94.39	**95.19**	91.14	83.32
	2	89.13	**89.64**	83.97	73.17
	3	71.60	**72.07**	62.50	51.81

*Table 3. Test accuracy (%) for all 15 corruption conditions (results/*_robustness.csv); best per row in bold.*

5.3 Effect of augmentation

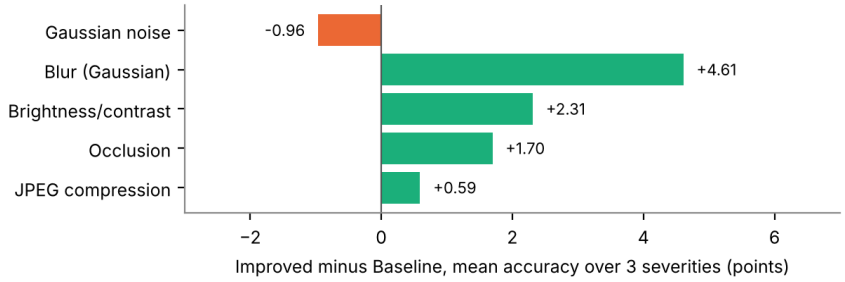


Figure 8. Change in mean accuracy from the baseline to the improved CNN. Gains are largest for blur (+4.6) and smallest or negative for noise (-1.0).

Augmentation improved four of five corruption types (blur +4.61, brightness/contrast +2.31, occlusion +1.70, JPEG +0.59 points) but slightly *reduced* accuracy under Gaussian noise (-0.96 points; at severity 3 64.32% vs 65.85%). The augmentation set contains no noise, so this is consistent with augmentation helping mainly for perturbations it resembles (jitter, rotation, scale). **Confound:** the baseline history has 8 epochs and the improved CNN 15, so the +1.65 point mCA gain cannot be attributed to augmentation alone; a baseline retrained for 15 epochs would resolve this and is listed as future work.

5.4 Honest comparison

- **Best overall:** improved CNN, ranked first on clean accuracy and mCA and on most corruption conditions in Table 3 (exceptions are noise severities 2-3 and occlusion/JPEG ties within about a point).
- **Pretrained is not better here:** ResNet-18 and MobileNetV2 lose 6-8 points of mCA relative to the scratch models in this setup, and 11.3M parameters (ResNet-18) buy 1.7 points *less* clean accuracy than the 0.82M-parameter improved CNN.
- **Noise is the weak point of transfer models:** at noise severity 3, MobileNetV2 reaches 24.16% and ResNet-18 39.75%, versus about 65% for the scratch CNNs. Because we only fine-tuned at 48x48 with the settings above, this does not show that pretraining is inherently fragile to noise.
- **Severity 3 of blur is the hardest condition** for every model (37-53%).

6. Error Analysis and Explainability

Per-class errors. From the per-class classification reports, the weakest classes of the improved CNN are "Double curve" (F1 0.714, recall 0.667), "Beware of ice/snow" (F1 0.936) and "Roundabout mandatory" (F1 0.941). The top-confused confusion matrix (Figure 9) shows 30 of the 90 "Double curve" test images predicted as "Right-of-way at next intersection", and 12 "General caution" images predicted as "Double curve". All three are warning triangles with small internal pictograms, which are the details that vanish at 32x32 resolution. "Double curve" is also among the lowest-F1 classes for ResNet-18 (0.760) and MobileNetV2 (0.798). For the baseline the lowest class is "Pedestrians" (F1 0.813), so augmentation shifted rather than removed the difficult classes, and it appears to have hurt "Double curve"; we did not investigate why (the class has only 90 test images, so the effect rests on few samples).

Model	Lowest-F1 classes (F1; recall)
Baseline CNN	Pedestrians (0.813; 0.833); End of speed limit (80km/h) (0.897; 0.813); End of all limits (0.902; 1.000)
Improved CNN	Double curve (0.714; 0.667); Beware of ice/snow (0.936; 0.880); Roundabout mandatory (0.941; 0.967)
ResNet-18	Speed limit (20km/h) (0.757; 0.650); Double curve (0.760; 0.756); Pedestrians (0.844; 0.950)
MobileNetV2	Beware of ice/snow (0.755; 0.607); Pedestrians (0.792; 0.667); Double curve (0.798; 0.722)

*Table 5. Three weakest classes per model by F1 on the test set (results/*_classification_report.csv).*

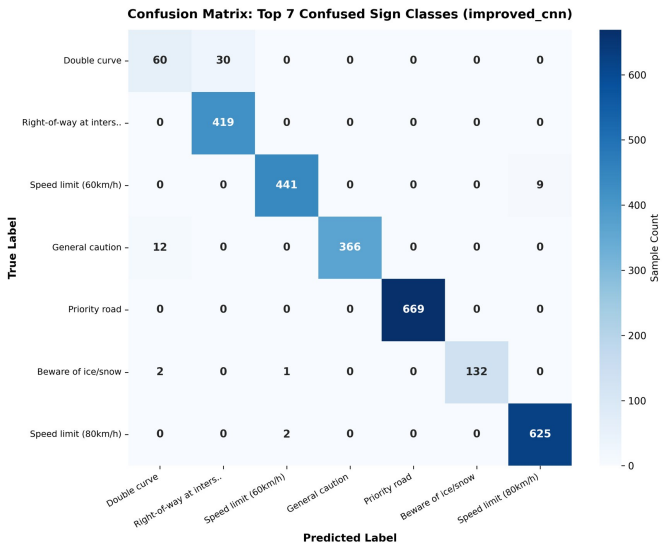


Figure 9. Confusion matrix restricted to the seven most confused classes, improved CNN (repository output).

Pretrained models fail differently. MobileNetV2 has high precision but low recall on "Beware of ice/snow" (1.00 / 0.607) and "Pedestrians" (0.976 / 0.667): it rarely predicts these classes, but misses many true instances. ResNet-18 misses "Speed limit (20km/h)" (recall 0.65), a class with only 60 test images that is visually close to other speed limits.

Grad-CAM. We implemented Grad-CAM natively in PyTorch (src/explainability.py) on the last convolutional layer of each model and generated heatmaps for clean samples, misclassifications and for each corruption (files under results/figures/). We make only limited claims from them. Grad-CAM maps are coarse (a few pixels wide at these input sizes), our inspection was on a small hand-picked set, and we did not quantify attention (for example with a deletion test or an inside-the-sign mass ratio). In the examples we inspected, the attention of a scratch CNN on a heavily blurred sign can shift to a small spot away from the sign even while the model stays highly confident, but this is one image and not a statistic. Moreover, the multi-model comparison figure uses a synthetic sign, and an earlier project document contained a statement that transfer models "are more robust"; this is contradicted by our measured results (Tables 2-3), so we do not carry it into this report. We treat Grad-CAM as a debugging aid and not as evidence of why a model is robust.

Real-world check (limited). The repository includes a 12-sign test with synthetic images drawn to resemble Indian signs and mapped to the nearest GTSRB class. Matches: baseline 6/12, improved CNN 5/12, ResNet-18 6/12, MobileNetV2 5/12. Several failures are expected from the setup itself: Indian speed-limit signs differ from German ones, and four classes (pedestrians, road work, slippery road, general caution) share the same generic drawn triangle. Average confidence over the 12 signs is 96-100% for every model although 6-7 of 12 were wrong, so confidence is a poor indicator of correctness here. Because the images are synthetic and the set is tiny, **this test cannot support any claim about real Indian roads.**

Demo. app.py (Gradio) lets a user upload a sign, choose a model, corruption and severity, and compare prediction, top-5 confidences and Grad-CAM for the clean and corrupted version. It reuses the same corruption and Grad-CAM code as the evaluation pipeline. Model weights are not stored in git; the app tells the user if a checkpoint is missing.

7. Limitations, Ethics and Future Work

Limitations.

- **Single seed, no confidence intervals.** Each model was trained once; differences below about one point are not reliable.

- **Training-length confound** between the baseline (8 epochs) and the improved CNN (15 epochs).
- **Synthetic corruptions.** Our five corruptions are simplified approximations of real degradation; the "blur" is isotropic Gaussian blur and not motion blur, and the corruptions are applied to the already-small GTSRB crops.
- **Low resolution.** Transfer models were run at 48x48, far below the 224x224 on which their backbones were pretrained. This may disadvantage them; a higher-resolution run was not tested.
- **Dataset scope.** GTSRB contains German signs only, in cropped form; detection of signs in a full scene is out of scope.
- **Grad-CAM analysis is qualitative** and the real-world test is synthetic (Section 6).

Ethics and safety. A traffic-sign model deployed in a vehicle can cause harm when it is wrong and confident; the high confidence we saw on wrong predictions argues for calibration, uncertainty estimates and fallback behaviour. The dataset contains German signs and may encode regional bias: a system trained on it would misread signs from other countries, as our small Indian-sign test illustrates. GTSRB contains no personal data beyond incidental scenery, but any extension that collects road photos should respect privacy. Our results must not be used to claim road-readiness.

Future work. (1) Retrain the baseline for 15 epochs and repeat each model with several seeds, reporting mean and standard deviation. (2) Add noise-style augmentation and test whether the noise gap closes. (3) Run transfer models at higher resolution. (4) Replace the synthetic Indian-sign check with a small set of real photographs, labelled by hand. (5) Quantify Grad-CAM (deletion/insertion tests) and add temperature scaling for calibration.

8. Conclusion

On GTSRB, a compact scratch-trained CNN with label-safe augmentation was the most accurate (98.70%) and the most robust (mCA 83.11%) of the four models, outperforming fine-tuned ResNet-18 (96.27%, 73.82%) and MobileNetV2 (93.54%, 66.18%) in our setup. Clean accuracy hid large differences: all models lose 15-27 points of accuracy on average under our corruptions, and the pretrained models are very sensitive to Gaussian noise. Augmentation helped for blur, brightness/contrast and occlusion but not for noise. The main caveats are single-seed training, the unequal training lengths of the two scratch models, and a qualitative explainability analysis. The central lesson is that robustness has to be measured explicitly, since clean accuracy alone would have suggested all four models are interchangeable.

9. Contribution Table

Member	Primary deliverables	Components owned (repository)	Suggested viva focus
Aseem (Leader)	Data acquisition, coordination, documentation, final report, demo UI	src/download_data.py, docs/, report/, README.md, app.py	Whole pipeline; report; Section 1-4
Raju	EDA, track split, baseline CNN, training loop, corruption bench	src/eda_and_split.py, models.py, train.py, dataset.py, corruptions.py, robustness_bench.py	Sections 3-5
Manoj	Transfer models, unified pipeline, two-stage fine-tuning	TransferModel in models.py, train.py rewrite, evaluate.py, run_phase4.py, run_all_phases.py	Sections 4-5.4
Sohail	Phase 4 training/evaluation, Grad-CAM and interpretation	src/evaluate.py, explainability / gradcam code, results/figures/	Sections 5-6

Table 4. Responsibilities taken from the repository README. Every member is expected to be able to explain every component; the last column is a suggestion for where each member should be strongest.

10. Declaration of AI-assistant Use

Claude (Anthropic) was used by the team leader to: read the repository and its results, draft the text of this report, produce Figures 1 and 3 and 5-8 from the committed CSV files, write app.py, and cross-check numbers against the code. Several statements in earlier repository documents (for example that baseline training used 15 epochs; that transfer models are "more robust"; older parameter counts; the title's 10.5x imbalance) disagreed with the committed results, and were replaced here by values computed from the files. Other members must add how they used AI assistants (for example for code generation or debugging) before submission.

11. References

[1] J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel. "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition." Neural Networks 32, 2012.

[2] P. Sermanet, Y. LeCun. "Traffic sign recognition with multi-scale convolutional networks." IJCNN, 2011.

[3] D. Hendrycks, T. Dietterich. "Benchmarking neural network robustness to common corruptions and perturbations." ICLR, 2019.

[4] K. He, X. Zhang, S. Ren, J. Sun. "Deep residual learning for image recognition." CVPR, 2016.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen. "MobileNetV2: Inverted residuals and linear bottlenecks." CVPR, 2018.

[6] R. R. Selvaraju et al. "Grad-CAM: Visual explanations from deep networks via gradient-based localization." ICCV, 2017.

[7] D. P. Kingma, J. Ba. "Adam: A method for stochastic optimization." ICLR, 2015.

[8] RoadSense code and results: https://github.com/rajukumar-tech/RoadSense-Robust-Traffic-Sign-Recognition

Appendix A. Supporting Figures



Figure A1. One example image from each of the 43 GTSRB classes (repository output).

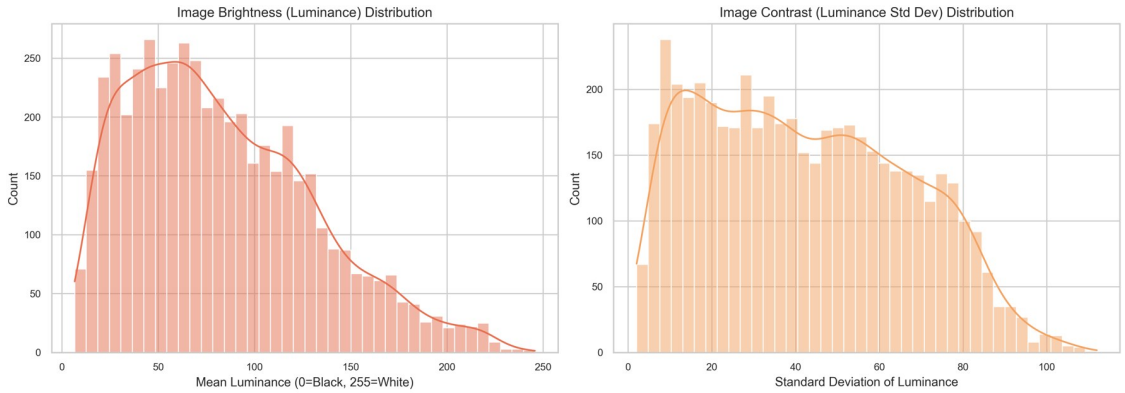


Figure A2. Brightness and contrast distributions of the training images: many images are dark (mean luminance mostly below 130), motivating the brightness/contrast corruption.